

Calibração da Rede Lunes — Taxas, Tempo de Bloco e Capacidade (TPS)

Este documento descreve as mudanças de parametrização aplicadas ao runtime (**spec_version 108**) para atingir os objetivos pedidos:

1. **Taxa de ~0,002 LUNES** para transações simples, com as demais operações em proporção equivalente;
2. **Tempo de bloco de 1 segundo**;
3. **Capacidade ≥ 10.000 TPS** para transferências simples;
4. **Requisitos de hardware** para operar a rede nessa configuração.

⚠ **Importante:** estas mudanças são de **parâmetros de consenso/econômicos** e **não foram compiladas nem testadas em cadeia** nesta máquina (sem toolchain Rust).

Elas **precisam** ser compiladas (`cargo build --release`), passar nos testes (`cargo test`) e ser validadas em **testnet** antes de qualquer uso em produção.

Os números de taxa/TPS abaixo são projeções calculadas a partir dos **pesos reais medidos** no benchmark (ver `docs/BENCHMARK.md`).

1. Mudanças aplicadas

Parâmetro	Antes (spec 107)	Depois (spec 108)	Arquivo
<code>MILLISECS_PER_BLOCK</code>	6000 (6 s)	1000 (1 s)	<code>runtime/src/constants.rs</code>
Orçamento de peso do bloco	$2 \times \text{WEIGHT_REF_TIME_PER_SECOND}$ (2 s)	$4 \times \dots$ (4 s)	<code>runtime/src/lib.rs</code>
<code>WEIGHT_FEE_DIVISOR</code>	(inexistente)	1380	<code>runtime/src/lib.rs</code>
<code>TransactionByteFee</code>	1 NANOUNIT (1000 planck)	10 planck	<code>runtime/src/lib.rs</code>
<code>spec_version</code>	107	108	<code>runtime/src/lib.rs</code>

O piso anti-spam `MINIMUM_WEIGHT_FEE = 1000` planck (correção da Fase 1) **foi mantido** — a taxa continua nunca podendo ser zero.

2. Objetivo 1 — Taxas calibradas (~0,002 LUNES)

A taxa de inclusão é `taxa_base + taxa_de_peso + taxa_de_tamanho`, onde as duas primeiras passam por `weight_to_fee(w) = max((ref_time + proof_size) / 1380, 1000)`.

O divisor **1380** foi escolhido para que uma transferência simples custe $\sim 0,002$ LUNES. Como a taxa é dominada pelo **peso**, as demais operações escalam **proporcionalmente**.

Operação	Peso (ref_time medido)	Taxa total	Em LUNES	Proporção
Transferência simples	174.250.000	200.055 planck	0,00200 LUNES	1,00×
Mint de NFT (nfts.mint)	575.248.000	490.683 planck	0,00491 LUNES	2,45×
Criar coleção NFT (nfts.create)	588.703.000	500.553 planck	0,00501 LUNES	2,50×
Chamada de contrato (con-tracts.call)	3.697.528.000	2.753.235 planck	0,02753 LUNES	13,76×
Deploy de contrato ~ 4 KB	7.279.668.548	5.389.659 planck	0,05390 LUNES	26,94×

(1 LUNES = 100.000.000 planck; taxa base fixa por extrínseco = $99.840.000/1380 \approx 72.348$ planck.)

Para reescalar TODAS as taxas mantendo a proporção, basta ajustar

`WEIGHT_FEE_DIVISOR` : dobrá-lo reduz as taxas pela metade, e vice-versa.

O depósito de armazenamento de contratos (reembolsável) é cobrado à parte pelo `pallet_contracts` e não entra nestes valores.

3. Objetivo 2 — Tempo de bloco de 1 segundo

`MILLISECS_PER_BLOCK` passou de 6000 para **1000**. As unidades de tempo derivadas (MINUTES , HOURS , DAYS , épocas, sessões) são definidas **em número de blocos** a partir de `SECS_PER_BLOCK` , então o **tempo de parede** de épocas/sessões/eras se mantém — apenas passam a conter 6× mais blocos.

⚠ A duração do slot não pode ser alterada em uma cadeia já em execução — fazê-lo trava a produção de blocos. Esta mudança só vale para uma **cadeia nova** (novo genesis) ou uma relançada de forma coordenada.

4. Objetivo 3 — Capacidade ≥ 10.000 TPS

Capacidade (transferências) = `orçamento_de_peso_normal / peso_por_transferência` , dividido pelo tempo de bloco.

Item	Valor
Orçamento de peso do bloco	4.000.000.000.000 (4 s de compute de referência)
Orçamento p/ extrínsecos normais (75 %)	3.000.000.000.000 (3 s)
Peso por transferência (base + dispatch)	274.090.000
Transferências por bloco	≈ 10.945
Tempo de bloco	1 s
TPS (teórico, blocos cheios)	≈ 10.945 TPS ✓

Assim, **na configuração de parâmetros o alvo de 10.000 TPS é atingido**. Porém, há uma restrição física de hardware que precisa ser entendida (seção 5).

5. Objetivo 4 — Requisitos de hardware (análise de viabilidade) ⚠

Esta é a parte mais importante e onde é preciso ser honesto.

5.1 O que o “peso” significa

No Substrate, `ref_time` é medido em **picossegundos de compute na máquina de referência** (`WEIGHT_REF_TIME_PER_SECOND = 1e12`). A máquina de referência do `polkadot-v0.9.40` é, aproximadamente, um **Intel Core i7-7700K (4 núcleos @ 4,2 GHz) com NVMe SSD**. Os pesos dos pallets (incluindo os ~174 M da transferência) foram calibrados nessa máquina — e são dominados por operações de **armazenamento** (RocksDbWeight: leitura ~25 M, escrita ~100 M).

5.2 A restrição fundamental (independente do tempo de bloco)

compute exigido por segundo = TPS × peso_por_transação

Para **10.000 TPS** de transferências:

$$\begin{aligned}
 10.000 \times 274.090.000 &= 2,74 \times 10^{12} \text{ ref_time por segundo de relógio} \\
 &= 2,74 \text{ segundos de compute de REFERÊNCIA por 1 s real}
 \end{aligned}$$

Ou seja, cada validador precisa **executar (e importar) 2,74 s de trabalho de referência a cada 1 segundo**. Reduzir o tempo de bloco de 6 s para 1 s **não muda** esse número — apenas divide o mesmo trabalho em blocos menores e mais frequentes.

Como a execução do FRAME clássico (v0.9.40) é **estritamente sequencial (single-thread)**, isso exige um único núcleo muito mais rápido que a referência:

Fração do slot usável p/ executar+importar	Speedup single-core exigido vs. referência
50 %	~5,5x
70 %	~3,9x

5.3 O problema

Os CPUs mais rápidos de 2024/2025 (ex.: Ryzen 9 7950X, Intel i9-14900K) têm desempenho **single-thread de ~2,0-2,5x** o i7-7700K. Isso está **abaixo** do ~3,9-5,5x necessário. O teto realista de single-thread fica em:

CPU (single-core vs ref.)	Utilização do slot	Teto realista de TPS
2,0x	50 %	~3.650 TPS
2,0x	70 %	~5.100 TPS
2,5x	50 %	~4.560 TPS
2,5x	70 %	~6.385 TPS

Conclusão honesta: com o stack atual (FRAME v0.9.40, execução sequencial, pesos de referência), **10.000 TPS sustentados não são fisicamente atingíveis** em hardware realista. Configurar o orçamento de peso para 4 s de compute em blocos de 1 s faz a cadeia **acompanhar apenas se** os validadores tiverem hardware ~4-5x referência; caso contrário, em blocos cheios a importação demora mais que o slot e a **cadeia trava**. O teto prático fica em **~4.000-6.000 TPS** para transferências simples, mesmo em hardware topo de linha.

5.4 Como realmente chegar a 10.000 TPS

Há três caminhos (idealmente combinados):

1. **Re-benchmark dos pesos em hardware moderno + ParityDB/NVMe.** Os pesos atuais são dominados por I/O de disco calibrado em RocksDB. Migrar para **ParityDB** em NVMe rápido e rodar `cargo build --features runtime-benchmarks + benchmark pallet` na máquina-alvo pode **reduzir o peso por transferência em ~2-3x**, dobrando/triplicando o teto de TPS.
2. **Mandato de hardware topo de linha** para validadores (ver 5.5) + orçamento de peso agressivo — aceitando **maior centralização** (poucos operadores conseguem participar).
3. **Migrar para o polkadot-sdk moderno** (ver `docs/MIGRATION_GUIDE.md`), que abre caminho para execução mais eficiente e futuras otimizações — é o caminho sustentável de longo prazo.

5.5 Requisito de hardware recomendado para esta configuração

Para operar um **validador** com a configuração agressiva (blocos de 1 s, orçamento de 4 s de peso) com a maior chance de acompanhar a rede:

Recurso	Mínimo (visando ~4-6k TPS reais)	Recomendado (tentar ~10k TPS)
CPU	8 núcleos, single-thread $\geq 2\times$ i7-7700K (ex.: Ryzen 7 7700X)	16 núcleos de altíssimo clock (Ryzen 9 7950X / i9-14900K / EPYC/Xeon de alto clock)
RAM	32 GB	64-128 GB
Disco	NVMe SSD 1 TB (ParityDB)	NVMe Gen4/Gen5 em RAID, alta IOPS, 2 TB+
Rede	1 Gbps simétrico, baixa latência	2,5-10 Gbps, latência mínima entre validadores
DB backend	<code>--database paritydb</code>	<code>--database paritydb + --state-pruning / --blocks-pruning</code> ajustados

Mesmo com o hardware “recomendado”, **10.000 TPS sustentados não são garantidos** com o runtime v0.9.40 sequencial; trate como um alvo a **validar empiricamente em testnet** com carga real, e não como número garantido.

6. Resumo

Objetivo	Situação	Observação
Taxa ~0,002 LUNES (transfer)	✅ Implementado	0,00200 LUNES; demais operações proporcionais
Demais taxas proporcionais	✅ Implementado	NFT 2,45×, contrato 13,76× — proporcionais ao peso
Tempo de bloco 1 s	✅ Implementado	Requer cadeia nova; slot não muda em cadeia viva
Capacidade ≥ 10.000 TPS (parâmetros)	✅ Implementado	~10.945 TPS teóricos com blocos cheios
Capacidade ≥ 10.000 TPS (física real)	⚠️ Não garantido	Teto realista ~4-6k TPS single-thread; precisa re-benchmark/ParityDB/migração
Requisito de hardware	✅ Documentado	Ver seção 5.5

Recomendação: compilar, rodar `cargo test`, subir uma **testnet** com este runtime e medir a capacidade real sob carga (com ParityDB no hardware-alvo). Se o objetivo de 10k TPS for firme, priorizar o **re-benchmark dos pesos** e a **migração para o polkadot-sdk**.